

Rapport de projet

Titre Professionnel Concepteur Développeur d'Applications

Lifequest - Application de gestion financière personnelle

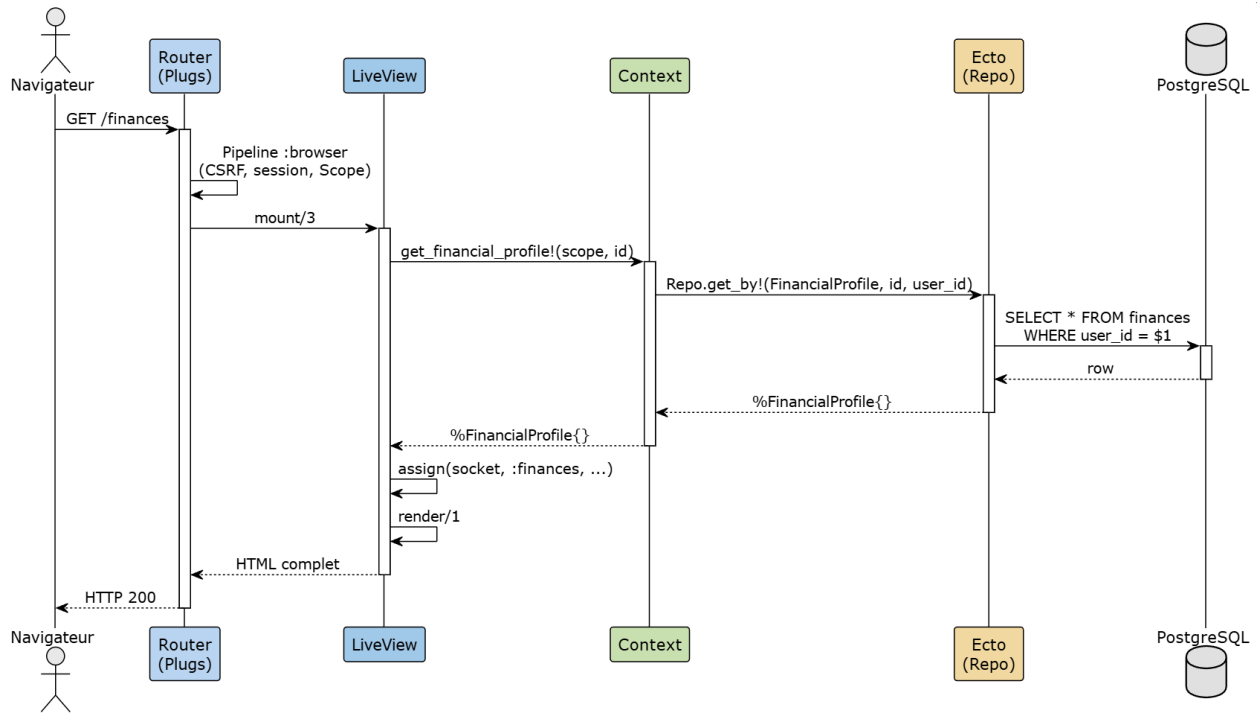
Candidat : Antonin MINGAM

Titre visé : Concepteur Développeur d'Applications Niveau 6

Référentiel : TP-01281 millésime 04 (24/05/2023)

Annexes

1.1 Diagramme de Séquence LiveView



2.1 Exemple d'utilisation de LiveView :

`dashboard_live`

```

defmodule LifequestWeb.DashboardLive.Index do
  use LifequestWeb, :live_view

  alias Lifequest.Finances

  @impl true
  def render(assigns) do
    ~H"""
    <Layouts.app flash={@flash} current_scope={@current_scope}>
      <h1 class="text-4xl font-bold mb-2">{gettext("Dashboard")}</h1>
      <p class="text-sm opacity-70 mb-8">
        {format_month(@current_month)}
      </p>

      <div class="grid grid-cols-1 md:grid-cols-2 gap-6 mb-12">
        <.donut_chart
          title={gettext("Monthly income")}
          slices={@income_slices}
          empty_label={gettext("No income this month.")}
        />
        <.donut_chart
          title={gettext("Monthly expenses")}
          slices={@expense_slices}
          empty_label={gettext("No expenses this month.")}
        />
      </div>

      <.pending_recurring_section direction={:income} pending={@pending_incomes}>
      <.pending_recurring_section direction={:expense} pending={@pending_expenses}>

      <.empty_state :if={@incomes == [] and @pending_incomes == []} direction={:income}>
      <.empty_state :if={@expenses == [] and @pending_expenses == []} direction={:expense}>

    </Layouts.app>
    """
  end
end

```

```

@impl true
def mount(_params, _session, socket) do
  scope = socket.assigns.current_scope
  today = Date.utc_today()

  {:ok,
   socket
   |> assign(:page_title, gettext("Dashboard"))
   |> assign(:current_month, today)
   |> load_dashboard_data(scope, today)}
end

defp load_dashboard_data(socket, scope, date) do
  incomes = Finances.list_transactions_for_month(scope, :income, date)
  expenses = Finances.list_transactions_for_month(scope, :expense, date)
  pending_incomes = Finances.list_pending_recurring(scope, :income, date)
  pending_expenses = Finances.list_pending_recurring(scope, :expense, date)

  socket
  |> assign(:incomes, incomes)
  |> assign(:expenses, expenses)
  |> assign(:pending_incomes, pending_incomes)
  |> assign(:pending_expenses, pending_expenses)
end

# --- Events ---

@impl true
def handle_event("validate_recurring", %{"id" => id}, socket) do
  scope = socket.assigns.current_scope
  transaction = Finances.get_transaction!(scope, id)
  date = socket.assigns.current_month

  case Finances.validate_recurring(scope, transaction, date) do
    {:ok, _new_transaction} ->

```

```

{:noreply, load_dashboard_data(socket, scope, date)}

{:error, _changeset} ->
  {:noreply, put_flash(socket, :error, gettext("Error validating t
end
end

defp pending_recurring_section(assigns) do
  ~H"""
  <div :if={@pending != []} class="card bg-warning/10 shadow mb-8">
    <div class="card-body">
      <h2 class="card-title mb-4">{pending_title(@direction)}</h2>
      <div class="space-y-3">
        <div :for={transaction <- @pending} class="flex justify-between">
          <div>
            <span class="font-medium">{transaction.label}</span>
            <span class="badge badge-outline badge-sm ml-2">
              {format_transaction_type(transaction)}
            </span>
          </div>
          <div class="flex items-center gap-3">
            <span>{format_currency(transaction.amount)}</span>
            <button
              phx-click="validate_recurring"
              phx-value-id={transaction.id}
              class="btn btn-primary btn-sm"
            >
              {gettext("Validate")}
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
  """
end

```

end
end

2.2 Exemple de tests unitaires Frontend :

dashboard_live_test


```

defmodule LifequestWeb.DashboardLive.IndexTest do
  use LifequestWeb.ConnCase

  import Phoenix.LiveViewTest
  import Lifequest.FinancesFixtures

  setup :register_and_log_in_user

  defp create_recurring_last_month(scope, attrs \\ %{}) do
    last_month = Date.shift(Date.utc_today(), month: -1)

    transaction_fixture(
      scope,
      Map.merge(
        %{
          label: "Recurring salary",
          direction: :income,
          income_type: :salary,
          amount: "3000.00",
          date: Date.new!(last_month.year, last_month.month, 5),
          is_recurring: true,
          is_active: true
        },
        attrs
      )
    )
  end

  describe "Dashboard with recurring transactions" do
    test "shows pending recurring incomes from last month", %{conn: conn} do
      create_recurring_last_month(scope)

      {:ok, _live, html} = live(conn, ~p"/dashboard")

      assert html =~ "Revenus récurrents à valider"
    end
  end
end

```

```

    assert html =~ "Recurring salary"
    assert html =~ "3000.00 €"
    assert html =~ "Valider"
end

test "shows pending recurring expenses from last month", %{conn: conn} do
    last_month = Date.shift(Date.utc_today(), month: -1)

    create_recurring_last_month(scope, %{
        label: "Recurring rent",
        direction: :expense,
        income_type: nil,
        expense_type: :essential,
        amount: "800.00",
        date: Date.new!(last_month.year, last_month.month, 5)
    })

    {:ok, _live, html} = live(conn, ~p"/dashboard")

    assert html =~ "Dépenses récurrentes à valider"
    assert html =~ "Recurring rent"
end

test "does not show recurring already validated this month", %{conn: conn} do
    create_recurring_last_month(scope)
    create_income(scope, %{label: "Recurring salary", amount: "3000.00"})

    {:ok, _live, html} = live(conn, ~p"/dashboard")

    refute html =~ "Revenus récurrents à valider"
end

test "validate_recurring duplicates transaction to current month", %{conn: conn} do
    create_recurring_last_month(scope)

    {:ok, live_view, _html} = live(conn, ~p"/dashboard")

```

```

    assert has_element?(live_view, "button", "Valider")

    live_view
    |> element("button", "Valider")
    |> render_click()

    html = render(live_view)

    refute html =~ "Revenus récurrents à valider"
    assert html =~ "3000.00 €"
  end
end
end

```

3.1 Pattern matching : clauses multiples et gestion des retours

```

defp format_transaction_type(%{direction: :income, income_type: type}),
  do: format_income_type(type)

defp format_transaction_type(%{direction: :expense, expense_type: type})
  do: format_expense_type(type)

defp format_income_type(:salary), do: gettext("Salary")
defp format_income_type(:freelance), do: gettext("Freelance")
defp format_income_type(:rental), do: gettext("Rental")
defp format_income_type(:bonus), do: gettext("Bonus")
defp format_income_type(_), do: gettext("Unknown")

```

```
def handle_event("validate_recurring", %{"id" => id}, socket) do
  scope = socket.assigns.current_scope
  transaction = Finances.get_transaction!(scope, id)
  date = socket.assigns.current_month

  case Finances.validate_recurring(scope, transaction, date) do
    {:ok, _new_transaction} ->
      {:noreply, load_dashboard_data(socket, scope, date)}

    {:error, _changeset} ->
      {:noreply, put_flash(socket, :error, gettext("Error validating tra
  end
end
```

3.2 Tests unitaires du contexte **Finances** : isolation par scope

```
describe "transactions" do
  import Lifestream.AccountsFixtures, only: [user_scope_fixture: 0]
  import Lifestream.FinancesFixtures

  test "list_transactions/1 returns all scoped transactions" do
    scope = user_scope_fixture()
    other_scope = user_scope_fixture()
    transaction = transaction_fixture(scope)
    other_transaction = transaction_fixture(other_scope)

    assert [result] = Finances.list_transactions(scope)
    assert result.id == transaction.id

    assert [other_result] = Finances.list_transactions(other_scope)
    assert other_result.id == other_transaction.id
  end

  test "get_transaction!/2 returns the transaction with given id" do
    scope = user_scope_fixture()
    transaction = transaction_fixture(scope)
    other_scope = user_scope_fixture()

    assert Finances.get_transaction!(scope, transaction.id) == transaction

    assert_raise Ecto.NoResultsError, fn ->
      Finances.get_transaction!(other_scope, transaction.id)
    end
  end
end
```

3.3 Définition d'une fixture : Transaction

```
``elixir def transaction_fixture(scope, attrs \ %{}) do account_id =  
Map.get(attrs, :account_id) || account_fixture(scope).id
```

```
  attrs =  
    attrs  
    |> Map.put_new(:account_id, account_id)  
    |> Enum.into(%{  
      amount: "120.50",  
      date: ~D[2026-03-05],  
      direction: :income,  
      income_type: :salary,  
      is_active: true,  
      is_recurring: false,  
      label: "some label"  
    })
```

```
  {:ok, transaction} = Lifequest.Finances.create_transaction(scope, attrs)  
  transaction
```

```
end ``
```

4.1 Plan de tests

Contexte **Finances** : Tests unitaires

ID	Fonctionnalité	Précondition	Action
T01	<code>list_transactions/1</code> : cas nominal	1 transaction créée pour le scope actif	Appeler <code>list_transactions(scope)</code>
T02	<code>list_transactions/1</code> : isolation scope	1 transaction par utilisateur	Appeler avec <code>scope</code> et <code>other_scope</code> séparément
T03	<code>get_transaction!/2</code> : cas nominal	1 transaction créée pour le scope	Appeler <code>get_transaction!(scope,</code>
T04	<code>get_transaction!/2</code> : isolation scope	Transaction créée pour <code>scope</code>	Appeler <code>get_transaction!(other_s id)</code>
T05	<code>create_transaction/ 2</code> : données valides	Compte existant pour le scope	Appeler avec attributs complets (label, direction, income_type, amount, date)
T06	<code>create_transaction/ 2</code> : données invalides	Scope existant	Appeler avec <code>@invalid_attrs</code> (tous l champs à <code>nil</code>)
T07	<code>update_transaction/ 3</code> : données valides	Transaction existante	Mettre à jour label, direction, amount
T08	<code>update_transaction/ 3</code> : scope invalide	Transaction créée pour <code>scope</code>	Appeler <code>update_transaction(other_scope transaction, %{})</code>
T09	<code>update_transaction/ 3</code> : données invalides	Transaction existante	Appeler avec <code>@invalid_attrs</code> puis r en base

ID	Fonctionnalité	Précondition	Action
T10	<code>delete_transaction/2</code> : cas nominal	Transaction existante	Appeler <code>delete_transaction(scope, transaction)</code>
T11	<code>delete_transaction/2</code> : scope invalide	Transaction créée pour <code>scope</code>	Appeler <code>delete_transaction(other_scope, transaction)</code>
T12	<code>sum_all_by_category/2</code> : totaux corrects	2 transactions salary + 1 freelance	Appeler <code>sum_all_by_category(scope, :income)</code>
T13	<code>sum_all_by_category/2</code> : catégorie absente	1 transaction salary	Vérifier la clé <code>:freelance</code> dans la map retournée
T14	<code>project_savings/2</code> : sans profil	Aucun profil financier pour le scope	Appeler <code>project_savings(scope, :income)</code>
T15	<code>project_savings/2</code> : avec récurrents	Profil + 1 revenu récurrent 500€ + 1 dépense 200€	Projeter sur 3 mois depuis 1000€ d'épargne
T16	<code>project_savings/2</code> : dette partiellement soldée	Profil avec 150€ de dettes, mensualité 100€	Projeter sur 3 mois
T17	<code>delete_all_user_data/1</code> : suppression complète	Profil, compte et transaction créés	Appeler <code>delete_all_user_data(scope)</code>
T18			Supprimer les données de <code>scope</code>

ID	Fonctionnalité	Précondition	Action
	delete_all_user_data/ 1 : isolation scope	Données pour scope et other_scope	

LiveView : Tests d'intégration

ID	Fonctionnalité	Précondition	Action	Résultat attendu
T19	Dashboard : rendu	Utilisateur connecté	Naviguer vers <code>/dashboard</code>	La page s'affiche
T20	Dashboard : redirect non authentifié	Connexion anonyme	<code>live(build_conn(), ~p"/dashboard")</code>	Redirection vers <code>users/login</code>
T21	Dashboard : revenus récurrents à valider	Transaction récurrente créée le mois précédent	Monter le Dashboard	La section "Revenus récurrents à valider" s'affiche et le montant est correct
T22	Dashboard : validation d'un récurrent	Revenu récurrent en attente affiché	Cliquer sur "Valider"	La section disparaît et le revenu apparaît dans les transactions validées du mois
T23	Dashboard : pas de doublons si déjà validé	Récurrent créé le mois dernier + déjà validé ce mois	Monter le Dashboard	La section disparaît et le revenu n'apparaît pas
T24	Formulaire transaction : rendu revenu	Utilisateur connecté, compte existant	Naviguer vers <code>/transactions/new?direction=income&income_type=salary</code>	Le titre "Nouveau revenu" s'affiche, les champs "direction" et "income_type" sont pré-remplis

ID	Fonctionnalité	Précondition	Action	Résultat attendu
				remplir le formulaire hidden
T25	Formulaire transaction : création revenu	Formulaire rendu	Soumettre le formulaire avec données valides	Rediriger vers la page financière avec le message "Transaction créée avec succès"
T26	Formulaire transaction : erreurs de validation	Formulaire rendu	Soumettre avec des champs vides	Les messages d'erreur peuvent apparaître "Champ vide" s'affichent de redirection
T27	Formulaire transaction : création dépense	Utilisateur connecté, compte existant	Soumettre un formulaire dépense valide	Rediriger vers la page financière avec confirmation
T28	Formulaire transaction : redirect non authentifié	Connexion anonyme	Naviguer vers <code>/transactions/new</code>	Rediriger vers la page users/index